in order to avoid excessive memory consumption. In this chapter you've seen three techniques for ensuring that memory can safely be reclaimed:

- Waiting until no threads are accessing the data structure and deleting all objects that are pending deletion
- Using hazard pointers to identify that a thread is accessing a particular object
- Reference counting the objects so that they aren't deleted until there are no outstanding references

In all cases the key idea is to use some method to keep track of how many threads are accessing a particular object and only delete each object when it's no longer referenced from anywhere. There are many other ways of reclaiming memory in lock-free data structures. For example, this is the ideal scenario for using a garbage collector. It's much easier to write the algorithms if you know that the garbage collector will free the nodes when they're no longer used, but not before.

Another alternative is to recycle nodes and only free them completely when the data structure is destroyed. Because the nodes are reused, the memory never becomes invalid, so some of the difficulties in avoiding undefined behavior go away. The downside here is that another problem becomes more prevalent. This is the so-called *ABA problem*.

7.3.3. Guideline: watch out for the ABA problem

The ABA problem is something to be wary of in any compare/exchange–based algorithm. It goes like this:

1. Thread 1 reads an atomic variable `x` and finds it has value `A`.
2. Thread 1 performs some operation based on this value, such as dereferencing it (if it's a pointer) or doing a lookup or something.
3. Thread 1 is stalled by the operating system.
4. Another thread performs some operations on `x` that changes its value to `B`.
5. A thread then changes the data associated with the value `A` such that the value held by thread 1 is no longer valid. This may be as drastic as